Part 1: build an agent you can explain

We are going to build the product launch analysis agent that can read materials, search, save outputs, and update a visible plan. You will choose its instructions and tools, then build one small action of your own. You will use the OpenAI Agents SDK directly to construct an agent and define tools. The supplied file-access and provider libraries handle infrastructure.

How the lab code fits together

`agent.py` defines the agent and its conversation input. The agent can call the functions in `tools.py`, which use `file_access/` to read and save files and `plans.py` to manage plans. `lab_support/` connects this code to the model and terminal. Start by editing the three files in the first three rows:

File or folder	Purpose	What you can change
agent.py	Defines instructions, available tools, the SDK `Agent`, conversation input, and the runner call	Edit `INSTRUCTIONS` and `TOOLS`; extend `prepare_input` to supply selected notes
tools.py	Defines the actions the agent can request	Modify a tool or add your own `@function_tool` function
try_tools.py	Calls a chosen tool with inputs you supply, without a model	Change imports, tool calls, and example arguments for testing a tool
plans.py	Defines plan fields, validation, saving, loading, and display	Read it for the plan exercise; extend it for a project that changes plan behavior
project_examples.py	Demonstrates a two-agent workflow and evaluation with scripted replies	Run it and adapt an example for a project
file_access/	Implements the Files API and its file-access rules	Use its public methods; leave the supplied implementation unchanged for the lab
`lab_support/`	Handles model connections, conversation history, terminal output, and offline testing	Leave unchanged for the lab; use its public extension interfaces
`data/`	Contains the fictional source materials	Read these; keep the supplied fixtures unchanged for the exercises
`runtime/`	Contains reports, notes, and plans saved by the live agent	Inspect outputs; use `update_plan` or `plans.save_plan` to change the plan
`.env.example` / `.env`	Provides a configuration template / holds your local key and model settings	Copy the template to `.env`, add your key, and keep the supplied model setting
`requirements.txt` / `pyproject.toml`	Defines the Python dependencies and package	Use for setup; leave unchanged for the lab
`sample_outputs/`	Shows example saved work	Read for comparison
`.lab-state/`	Stores usage accounting created by the program	Leave unchanged

EXERCISES.md guides the activities. EXTENDING.md explains how to build on the lab with memory, interfaces, workflows, and evaluation. PDF copies of this README, the exercises, and the extension guide are included for reading and printing.

Setup before class

Use Python **3.11 or newer**. Open a terminal in this lab folder (the folder containing `agent.py` and `requirements.txt`). In VS Code, open the lab folder, then choose **Terminal → New Terminal**. Enter commands one line at a time and press Enter after each line.

Create a virtual environment (`.venv`), which keeps this lab's Python packages together, then activate it and install the required packages.

Windows PowerShell:

```
py -3 --version
py -3 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
```

Check that the first command reports Python 3.11 or newer before continuing.

If activation is unavailable, use `.\.venv\Scripts\python.exe` in place of `python` in the install and run commands.

macOS / Linux:

```
python3 --version
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

Check that the first command reports Python 3.11 or newer before continuing.

First try the example tool:

```
python try_tools.py
```

You should see the input, a saved recommendation, and the result returned to the agent. This uses temporary files and makes no model or web requests. It works before you set up a key.

For the live agent, create a file named `.env` by copying `.env.example` in your editor. If `.env` already exists, edit it rather than replacing it. Paste the instructor-issued OpenRouter key into that file. Keep the supplied model setting. Do not paste your key into the agent conversation or submit it with your work.

```
python agent.py
```

Enter a task in the terminal. Type `/exit` to finish. You can optionally run `python agent.py --check` to check the remaining key balance; this does not run the model.

Run the code after setup

You only need to create `.venv`, install the packages, and configure `.env` once for this copy of the lab. When you return to the lab or open a new terminal:

1. Open a terminal in the lab folder, as above. If your terminal is elsewhere, run `cd "path/to/lab"`, replacing `path/to/lab` with the actual folder path. Keep the quotes if the path contains spaces.
2. Activate the existing environment using the command for your system:

   Windows PowerShell:

   ```
   .\.venv\Scripts\Activate.ps1
   ```

   macOS / Linux:

   ```
   source .venv/bin/activate
   ```

   You will usually see `(.venv)` at the start of the terminal prompt. Activation applies to this terminal; repeat it in each new terminal.
3. Choose what to run, and enter its command in that same terminal:

What you want to do	Command
Try the tool calls you wrote in `try_tools.py`	`python try_tools.py`
Start a conversation with the agent	`python agent.py`

`python` runs the `.py` file named after it. Enter these commands at the terminal prompt, not inside a Python >>> prompt or the agent conversation. If you see >>>, enter `exit()` to return to the terminal first.

4. When the agent asks for input, type a task and press Enter. For example: “List the files in data/.” Continue typing follow-up requests in the same conversation. Type `/exit` to stop the agent and return to the terminal.

After editing a Python file, save it in your editor. If the agent is running, type `/exit`, then run `python agent.py` again to load your changes. To try an edited tool directly, run `python try_tools.py` again. You do not need to recreate or reactivate the environment while using the same terminal.

If PowerShell blocks activation, run `.\.venv\Scripts\python.exe agent.py` or `.\.venv\Scripts\python.exe try_tools.py` directly. On macOS / Linux, the equivalent commands are `.venv/bin/python agent.py` and `.venv/bin/python try_tools.py`; these also work without activation.

Test different tools

Open try_tools.py, import the tool you want from `tools`, and put a `try_tool` call under `if __name__ == "__main__":`. Pass the tool itself as the first argument, followed by named arguments matching its definition in `tools.py`. For example, replace the starting recommendation call with:

```
from lab_support.testing import try_tool
from tools import search_local

if __name__ == "__main__":
    try_tool(search_local, query="import", path="data")
```

Run `python try_tools.py` again after each edit. The tester prints the tool name, input arguments, result returned to the agent, and the contents of any saved files. It also returns that result to your Python code.

These are other calls you can use; import each tool you choose from `tools`:

Call	What to look for
`try_tool(list_files, path="data")`	A dictionary listing accessible paths and whether the listing was truncated
`try_tool(read_file, path="data/customer-interviews.md")`	The file’s text as a string
`try_tool(write_file, path="runtime/question.md", content="Why is import difficult?")`	A confirmation string and a preview of the saved file
`try_tool(save_recommendation, recommendation="Test an import preview.")`	A confirmation and the contents saved to the fixed recommendation path
`try_tool(search_local, query="IMPORT", path="data")`	The same list of filenames as for `"import"`; try `"zzznomatchzzz"` to get `[]`

For a tool with structured input, pass ordinary Python lists and dictionaries:

```
try_tool(update_plan, goal="Understand onboarding", steps=[
    {"id": "s1", "description": "Read interviews", "status": "in_progress"},
    {"id": "s2", "description": "Compare needs", "status": "pending"},
], revision_note="Initial plan")
```

To test a new tool, import its name and supply the inputs you defined. It does not need to be in the agent’s `TOOLS` list for this test. Try a normal input and one that should fail, such as asking `write_file` to write inside `data/`, and compare the returned results with what you expected.

Each call starts fresh: it copies `data/` into a temporary folder and starts with no saved outputs. Your live `runtime/` files are not used or changed, and the temporary folder is removed after the call. A second `try_tool` call cannot read a file saved by the first. To test a write followed by a read, put both operations inside one custom tool, or test the sequence through the live agent. `search_web` requires a live connection, so test it through `agent.py`.

Some important ideas

Idea	Meaning in this lab
Instructions	Tell the agent its role and how to approach the user's task
Tools	Actions the agent can request, with inputs and returned results
Human intervention	You change direction or answer a question; enforced approval pauses come in the next lab

The basic sequence is:

Your request → agent chooses a tool and its inputs → Python runs the tool → the result goes back to the agent → agent continues or answers.

Python carries out the action. The model chooses when to request it. An available tool is not necessarily used on every turn.

Choose what your agent can do

In `agent.py`, `create_agent(model)` is an ordinary Python function containing the actual SDK constructor:

```
agent = Agent(
    name="Product analysis assistant",
    model=model,
    instructions=INSTRUCTIONS,
    tools=TOOLS,
)
```

The model generates responses, the instructions guide its behavior, and the tool list gives it available actions. Construction creates the agent object; it does not yet ask the model to do anything. Change the name, instructions, or tools here.

`run_agent` contains the standard SDK runner call:

```
result = Runner.run_sync(agent, messages, max_turns=12, run_config=run_config)
```

`messages` is the conversation input for this run. On the first turn it contains your request, represented as a list of messages:

```
messages = [{"role": "user", "content": "Read the customer interviews."}]
```

`role` identifies who is speaking; `content` contains the message text. On later turns, `prepare_input` in `agent.py` adds your new request to the saved history:

```
messages = history + [{"role": "user", "content": user_input}]
```

That history includes previous user and assistant messages as well as tool calls and their results. It lets the agent interpret follow-ups such as "compare those two ideas." The SDK supplies the tool records through `result.to_input_list()`; you do not need to construct them yourself. The agent's `INSTRUCTIONS` are set on the `Agent` separately. Files on disk are included only when your code supplies their contents or the agent reads them with a tool.

The runner sends the messages, executes requested tools, returns their results to the model, and continues until it has an answer or reaches the turn limit. `result.final_output` is the answer. `result.to_input_list()` gives the history for the next user turn. The terminal helper prints the answer and retains that history; `run_config` contains supplied provider and execution settings.

Importing a tool makes it available to your Python file; adding it to `TOOLS` makes it available to your agent. All seven definitions are visible in `tools.py`.

Ready-made tool	What it does
`list_files(path)`	Discover filenames in a folder, without reading contents
`read_file(path)`	Read a text file
`write_file(path, content)`	Save or replace a text file
`search_web(query)`	Find public sources and return excerpts and links

Ready-made tool	What it does
`search_local(query, path)`	Return filenames whose contents match the query in a local folder
`update_plan(goal, steps, revision_note)`	Save and display the current plan

The `save_recommendation` tool in `tools.py` is a custom example. It saves to one fixed filename. Compare that with `write_file`, where the agent chooses the path.

Try this first request:

> List the files in data/. Read the customer interviews and suggest one product question we could investigate. Don't search the web or save a report yet.

Then choose a question and ask for a plan. Open `runtime/plan.md` alongside the terminal. Later, change a requirement and see whether the plan changes.

Build a tool using the file library

Every tool in `tools.py` uses the standard SDK `@function_tool` decorator. The function has an input, a description, an action, and a result. The type `str` means text. The supplied **Files API** (`from file_access import files`) gives your Python tools a consistent way to read source materials and save outputs. It handles paths, UTF-8 text, folder creation, and file limits. It also enforces the lab's rules: read from `data/` and `runtime/`, write only to `runtime/`, and change plans through the plan functions. You can concentrate on what your tool does without repeating those checks in every tool.

Files matter because they connect the conversation to source evidence and work you can inspect and reuse. A read supplies material the model can reason about; a write creates an artifact that survives the session. For example:

```python
from agents import function_tool
from file_access import files

@function_tool
def save_recommendation(recommendation: str) -> str:
    """Save the recommendation after the user has agreed to it."""
    files.write_text("runtime/recommendation.md", recommendation)
    return "Recommendation saved to runtime/recommendation.md."
```

The model sees the name, description, and input definition. When it requests the tool, Python runs its body. The `return` value goes back to the model.

These library calls run inside your Python tool. Paths are strings relative to the lab folder, such as `"data/customer-interviews.md"`, not absolute paths. Use `/` as the separator on every operating system.

Library call	Input	Output and effect
`files.read_text(path)`	Path to one existing `.md` or `.txt` file in `data/` or `runtime/`	Returns the entire UTF-8 text as a `str`; raises an error if missing or rejected
`files.write_text(path, text)`	Destination `.md` or `.txt` path under `runtime/` and a string to save	Creates parent folders and saves or replaces the file; returns `None`. Raises an error if rejected
`files.list_files(path=".")`	A folder in `data/` or `runtime/`; `"."` discovers those two folders	Returns a dictionary with `path`, `entries`, `truncated`, and `message`. Lists immediate children only, up to 64, without reading contents
`files.read_texts(path)`	A folder such as `"data"` or `"runtime/notes"`	Returns a `dict[str, str]` mapping paths to complete text, including subfolders. Returns `{}` for a missing or empty folder; `"."` is not accepted

For example, after writing a single note to an otherwise empty folder:

```
files.write_text("runtime/notes/question.md", "Why is import difficult?")
text = files.read_text("runtime/notes/question.md")
# text == "Why is import difficult?"

documents = files.read_texts("runtime/notes")
# {"runtime/notes/question.md": "Why is import difficult?"}

listing = files.list_files("runtime/notes")
# {
#     "path": "runtime/notes",
#     "entries": [{"path": "runtime/notes/question.md", "type": "file"}],
#     "truncated": False,
#     "message": "Accessible entries only; no file contents read."
# }
```

An entry's `type` is `"file"` or `"directory"`. Use its `path` directly in a subsequent read or listing. Reads and writes allow at most 32 KiB per file; folder reads allow at most 64 files and 256 KiB of text. Generated files share a 64-file, 256 KiB limit. A truncated listing is flagged; a read that exceeds the limits raises an error instead of silently returning partial text.

Calling a file helper inside your tool does not give the model a separate tool. You choose what inputs your tool exposes and what result it returns. For example, `write_text` returns `None`, so `save_recommendation` returns its own confirmation string after saving. Terminal messages are for you to inspect; your tool's `return` value is what the agent receives. See the Files API reference for more details.

The file library also works by itself, without an agent:

```
from file_access import FileAccess

files = FileAccess("my_project")
files.write_text("runtime/question.md", "What makes onboarding difficult?")
question = files.read_text("runtime/question.md")
```

`FileAccess` uses the same `data/` and `runtime/` conventions for any chosen root. For a new project, keep the standard SDK `Agent`, `Runner`, and `@function_tool` pattern. Supply that project's model and a file library appropriate to its needs.

Search and plans

`search_local(query, path)` combines `files.read_texts(path)` with a matching loop that students can write themselves. Choose `data` for supplied sources, `runtime` for generated work, or `runtime/notes` for saved notes. It searches for the entire query as one substring using `lower()` and `in`, then returns up to five distinct filenames in alphabetical order. Matching ignores case and allows partial words: `"import"` also matches `"imported"`. Spaces, line breaks, and punctuation must match exactly: `"import preview"` does not match `"import\npreview"`. Empty or whitespace-only queries return `[]`. For example, a result might be `["runtime/notes/interview.md", "runtime/notes/pilot.md"]`. The agent can pass a returned path to `read_file` to inspect the evidence. Missing folders, empty queries, and searches without matches return `[]`.

`PlanStep` in plans.py defines `id`, `description`, and the four allowed statuses. `update_plan` in `tools.py` calls the shared plan functions. The complete input looks like this:

```
{
  "goal": "Understand onboarding",
  "steps": [
    {"id": "s1", "description": "Read interviews", "status": "in_progress"},
    {"id": "s2", "description": "Compare needs", "status": "pending"}
  ],
  "revision_note": "Initial plan"
}
```

Each call replaces the whole plan. Reuse IDs for continuing steps. `save_plan` in `plans.py` checks for 1–8 steps, unique IDs, and at most one step in progress. It saves structured fields in `runtime/plan.json` and a readable `runtime/plan.md`. The tool then displays the plan. Invalid updates preserve the previous plan. A `done` status is a claim by the agent, so compare it with evidence that the work succeeded. Scripts and future interfaces can use the same plan functions.

Files, memory, and rules

`data/` contains supplied materials. `runtime/` contains generated outputs. Both are ordinary folders you can open in your editor. File tools read `.md` and `.txt` files in those folders and write only inside `runtime/`. The plan file is reserved for `update_plan`. The program enforces these limits even if an instruction asks the agent to ignore them.

Saved files survive exit. The conversation does not: after restarting, ask the agent to read the files it needs. Saving something does not automatically make it available in every later model input.

If something goes wrong

- **Missing module:** activate your environment and install `requirements.txt`.
- **Environment copied from another computer:** create a fresh environment for this computer; virtual environments are not portable.
- **Missing key:** check your local `.env` and the instructor's setup instructions.
- **File rejected:** check the folder, extension, and whether the path is the plan.
- **Search or turn limit:** inspect saved files and continue with a focused request.
- **API unavailable:** use `python try_tools.py` or `python agent.py --demo`. The demo uses scripted responses and requires the six ready-made tools in `TOOLS`. It illustrates the loop; it cannot evaluate your instruction edits.

Continue with the exercises.

Build on the lab

See Start extending your agent for small starting points and optional paths into plans, memory, interfaces, multiple agents, and evaluation. The required editing path remains `agent.py`, `tools.py`, and `try_tools.py`.